

# Lijo Technologies Universal-Gateway

User Guide-tested and modified

June 2024

# Summary

Summary.....

Overview.....

1. Scripts (py).....

1.1. Adding Device Profiles Script.....

1.1.1. Script Overview.....

1.1.2. Prerequisites.....

1.1.3. File Preparation.....

1.1.4. Running the Script.....

1.1.5. Script Execution.....

1.1.6. Troubleshooting.....

1.2. Adding Devices Script.....

1.2.1. Script Overview.....

1.2.2. Prerequisites.....

1.2.3. File Preparation.....

1.2.4. Running the Script.....

1.2.5. Script Execution.....

1.2.6. Troubleshooting.....

1.3. Adding ONVIF Devices Script.....

1.3.1. Script Overview.....

1.3.2. File Preparation.....

1.3.3. Running the Script.....

1.3.4. Script Execution.....

1.4. Adding Stream Script.....

1.4.1. Script Overview.....

1.4.2. File Preparation.....

1.4.3 Running the Script.....

1.4.4. Script Execution.....

1.5. Adding Rule Script.....

1.5.1. Script Overview.....

1.5.2. File Preparation.....

1.5.3. Running the Script.....

1.5.4. Script Execution.....

1.6. Command Device Script.....

1.6.1. Script Overview.....

1.6.2. File Preparation.....

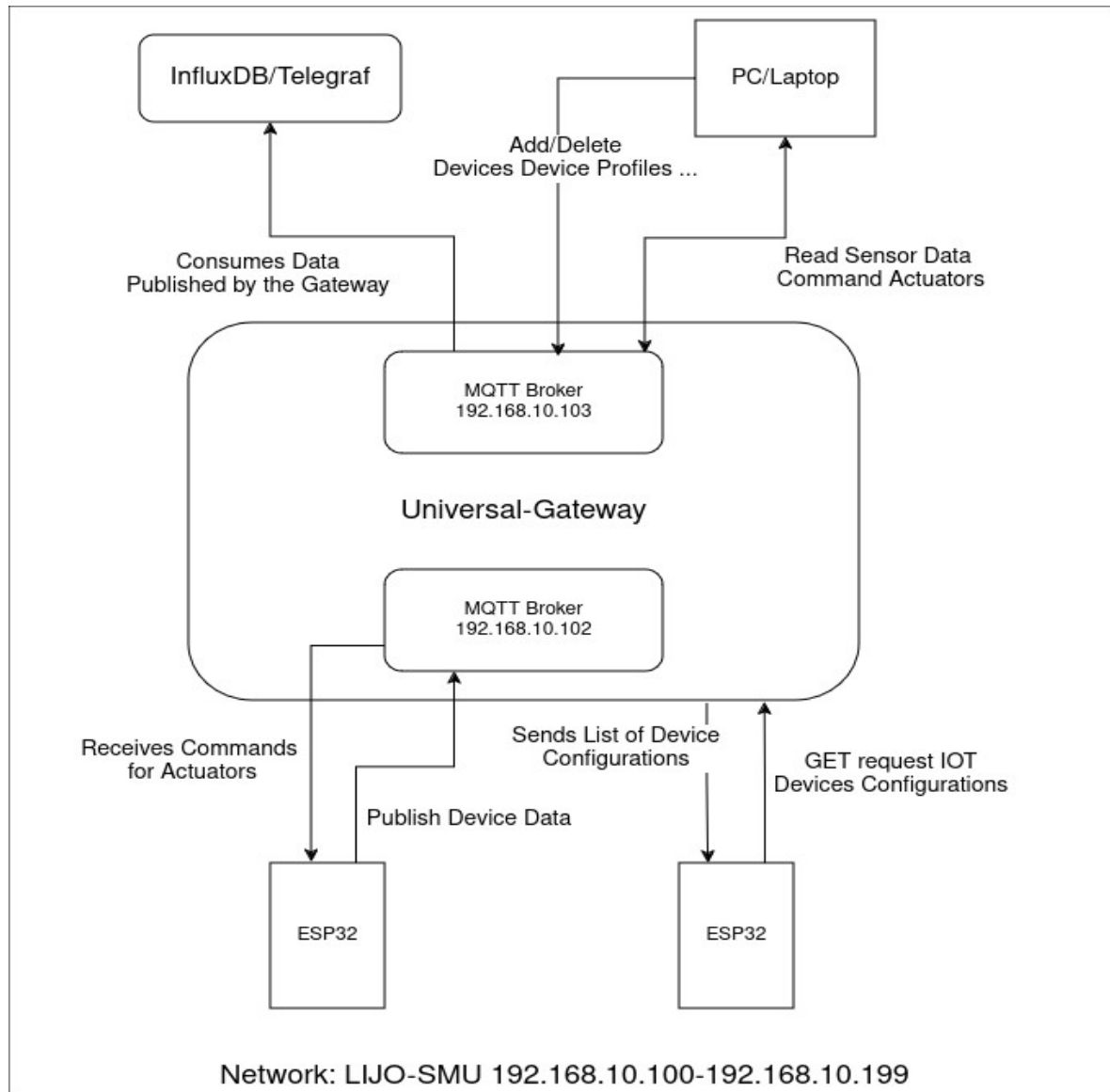
1.6.3. Running the Script.....

1.6.4. Script Execution.....

1.7. Delete Device Script.....

|                                             |  |
|---------------------------------------------|--|
| 1.7.1. Introduction.....                    |  |
| 1.7.2. How to Use.....                      |  |
| <b>1.8. Delete All Devices Script.....</b>  |  |
| 1.8.1. Introduction.....                    |  |
| 1.8.2. How to Use.....                      |  |
| <b>1.9. Delete Profile Script.....</b>      |  |
| 1.9.1. Introduction.....                    |  |
| 1.9.2 How to Use.....                       |  |
| 1.10. Delete Devices by Profile Script..... |  |
| 1.10.1. Introduction.....                   |  |
| 1.10.2. How to Use.....                     |  |
| 1.11. Delete Rule Script.....               |  |
| 1.11.1. Introduction.....                   |  |
| 1.11.2. How to Use.....                     |  |
| 1.12. Delete Stream Script.....             |  |
| 1.12.1. Introduction.....                   |  |
| 1.12.2. How to Use.....                     |  |
| <b>2. Tutorial.....</b>                     |  |
| 2.1. Adding new Sensor.....                 |  |
| 2.1.1. Adding Sensor code.....              |  |
| 2.1.2. Adding Device Profile.....           |  |
| 2.1.3. Adding Device.....                   |  |

# Overview



| App                          | IP Address     | Port  |
|------------------------------|----------------|-------|
| MQTT Broker (for ESP32)      | 192.168.10.102 | 31883 |
| Configs Server*              | 192.168.10.102 | 30300 |
| MQTT Broker (gateway export) | 192.168.10.103 | 31883 |
| InfluxDB                     | 192.168.10.103 | 30344 |

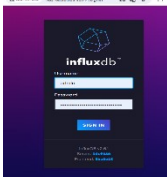
\* The URL used to get the configs list is "http://192.168.10.102:30300/configs".

| App/Component   | Username | Password                             |
|-----------------|----------|--------------------------------------|
| InfluxDB        | admin    | qUHgAl0oXLT6x3E2yZ8dUK<br>5O2BtP95rc |
| LIJO-SMU (Wifi) |          | 3@7984A5B                            |

# Important Instructions

## 1. Ensuring InfluxDB Connectivity

### Connect to InfluxDB:



- o Open your browser.
- o Enter the IP address and port assigned to your InfluxDB instance.

### Troubleshooting:

- o If you receive a **"Unable to connect"** message, please contact the Cloud Society support team for assistance.

## 2. Accessing the MQTT Broker

Note that the MQTT broker cannot be accessed directly via a browser.

- o Install an MQTT client tool to connect to the broker.
- o Example: Install MQTT Explorer using the following command

```
$ Sudo snap install mqtt-explorer
```

## 3. Installing python:

```
$ sudo apt update  
$ Sudo apt install python3
```

Verify installation:

```
$ python3 --version
```

## 4. For Uploading and Monitoring the ESP32

### Install Drivers :

- o CP210x:

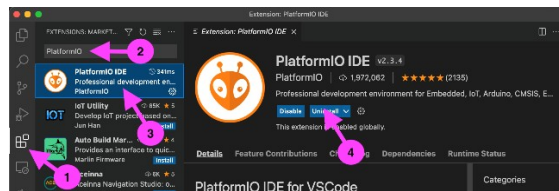
<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>

- o **Troubleshooting for linux: virtual COM Port Drivers**

<https://www.bordergate.co.uk/configuring-an-esp32-in-ubuntu-22-04/>

### Install PlatformIO Extension:

- o Through VS Code:



- o Through Terminal: by using the following commands :

- Step 1: Install PlatformIO Core

```
$ pip install platformio
```

- Step 2: Verify Installation

```
$ pio --version
```

- Step 3: install platform ide extension in vs

**OR:**

**<https://docs.platformio.org/en/latest/core/installation/methods/installer-script.html#installation-installer-script>**

# 1. Scripts (py)

## 1.1. Adding Device Profiles Script

This script automates the process of adding device profiles to the Universal Gateway via MQTT by publishing the necessary metadata.

### 1.1.1. Script Overview

The `add_profile.py` script facilitates the addition of device profiles to the Gateway. It reads device profile details from a JSON file and publishes this information to an MQTT broker. The IoT-Config-Server, subscribed to the appropriate MQTT topic, consumes this metadata to add the profiles to the Gateway Metadata, enabling device management through these profiles.

### 1.1.2. Prerequisites

This script can be run from your machine. Connect your Ubuntu machine to the “LIJO-SMU” network.

Then, ensure the following requirements are met:

- **Python Installation:** Python 3.x must be installed on your system.
- **Paho-MQTT Library:** The script uses the Paho-MQTT client. Install it using pip:  
bash

```
$ pip install paho-mqtt
```

### Error1: Pip not found

```
ubuntu@ubuntu-VirtualBox:~$ pip install paho-mqtt
Command 'pip' not found, but can be installed with:
sudo apt install python3-pip
ubuntu@ubuntu-VirtualBox:~$ ^C
ubuntu@ubuntu-VirtualBox:~$ sudo apt install python3-pip
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

### Error2: externally-managed-environment

```
ubuntu@ubuntu-VirtualBox:~$ pip install paho-mqtt
error: externally-managed-environment

This environment is externally managed.
```

## Installed with

```
$ sudo apt install python3-paho-mqtt
```

### 1.1.3. File Preparation

You need two JSON files:

- **device\_profile.json**: Contains the metadata for the device profiles to be added.
- **variables.json**: Holds the MQTT broker connection details.

The `device_profile.json` file contains the definitions for device profiles. Each device profile specifies the attributes and capabilities of a type of device managed by the Universal Gateway. Here's an in-depth look at the components of a device profile:

#### Example Device Profile: Temperature\_Humidity DHT11

```
{
  "DeviceProfile": [
    {
      "name": "Temperature_Humidity_DHT11",
      "manufacturer": "XXX",
      "model": "DHT11",
      "labels": [
        "esp32",
        "sensor",
        "mqtt",
        "temperature",
        "humidity"
      ],
      "description": "Temperature Humidity Sensor",
      "deviceResources": [
        {
          "name": "temperature",
          "isHidden": false,
          "description": "Temperature in Celsius",
          "properties": {
            "valueType": "Int8",
            "readWrite": "R",
            "units": "Celsius"
          }
        },
        {
          "name": "humidity",
          "isHidden": false,
          "description": "Humidity in percent",
```

```

        "properties": {
            "valueType": "Float32",
            "readWrite": "R",
            "units": "percent"
        }
    ]
}

```

### Fields Explained:

- **name:** Unique identifier for the device profile. This should be a clear and descriptive name.
- **manufacturer:** The manufacturer of the device. This can help identify the device and often affects compatibility and available resources.
- **model:** The model number or identifier provided by the manufacturer.
- **labels:** A list of descriptive labels or tags that provide additional context about the device's functions or environment. These can be used for searching or categorizing devices.
- **description:** A brief description of the device. This should provide a quick overview of what the device is and its primary functions.
- **deviceResources:** A list of resources or capabilities that the device offers. Each resource defines a specific data point the device can provide or accept.

### Inside deviceResources:

- **name:** The name of the resource, which should clearly identify what it represents (e.g., "temperature").
- **isHidden:** A boolean indicating whether the resource should be hidden in user interfaces. Usually, this is set to `false` to ensure visibility.
- **description:** A brief description of what the resource represents or its purpose (e.g., "Temperature in Celsius").
- **properties:** Defines the characteristics of the resource.
  - **valueType:** The data type of the resource (e.g., `Int8`, `Float32`). This should match the type of data the device outputs.
  - **readWrite:** Specifies whether the resource can be read (**R**), written (**W**), or both (**RW**). For sensors, this is typically set to **R**.
  - **units:** The units in which numeric values are measured (e.g., Celsius, percent).

When setting up a new device profile or modifying an existing one, it's crucial to ensure all fields are correctly defined according to the capabilities and requirements of the device. This

configuration determines how the Universal-Gateway interacts with the device, affecting data collection, command execution, and data display.

#### Example: variables.json

```
{
  "mqttBroker": "192.168.10.103",
  "mqttPort": 1883,
  "mqttTopicNewDeviceProfile": "Devices/ESP1/Config/NewOnvifDevice"
}
```

- **mqttBroker**: MQTT broker host
- **mqttPort**: MQTT broker port
- **mqttTopicNewDeviceProfile**: Topic used to add new device profiles, subscribed to by the IoT-Config-Server.

**This is only an example and you need to update it according to your network**

#### 1.1.4. Running the Script

To execute the script, use the following command:

```
$ python3 add_profile.py
```

The `device_profile.json` file needs to be in the same folder as the `add_profile.py` script.

- **You need to run the script in the directory where the file is stored**
- **Each time you add a profile you need to update the script**

#### 1.1.5. Script Execution

Upon running the script, it connects to the MQTT broker and publishes each device profile's metadata to the specified topic. The console will display a message for each profile added:

```
Added Device Profile: [Profile Name]
```

#### 1.1.6. Troubleshooting

If you encounter issues, consider the following common problems and solutions:

- **MQTT Connection Failure**: Ensure the MQTT broker address and port are correct in `variables.json`. Check network connectivity.

- **JSON Format Error:** Verify that the `devices_list.json` and `variables.json` files are properly formatted. Use a JSON validator if necessary.

#### Example JSON Format Error:

```
ubuntu@ubuntu-Virtual-Machine:~/Desktop/test$ python3 add_profile.py
Traceback (most recent call last):
  File "/home/ubuntu/Desktop/test/add_profile.py", line 25, in <module>
    for profile in device_profile_data['deviceProfiles']:
    ~~~~~^~~~~~
KeyError: 'deviceProfiles'
```

The error message indicates that there's a `KeyError` because the key `'deviceProfiles'` is not found in the `device_profile_data` dictionary. This suggests that the JSON file you're loading doesn't contain a key named `'deviceProfiles'`. à **You are just messing an array (deviceProfile array)**

- **Script Fails to Run:** Make sure all required libraries are installed. Check for typos or errors in the command line used to run the script.

## 1.2. Adding Devices Script

This script automates the process of adding new devices to the Universal-Gateway via MQTT by publishing the necessary metadata.

### 1.2.1. Script Overview

The “**add\_device\_from\_json.py**” script facilitates the bulk addition of IoT devices to the Universal-Gateway. It reads device details from a JSON file and publishes this information to an MQTT broker, where the Gateway is a subscriber.

### 1.2.2. Prerequisites

Before running the script, ensure the following requirements are met:

- **Python Installation:** Python 3.x must be installed on your system.
- **Paho-MQTT Library:** The script uses the Paho-MQTT client. Install it using pip:  
bash

```
$ pip install paho-mqtt
```

### 1.2.3. File Preparation

You need two JSON files:

- **devices\_list.json**: Contains the metadata for the devices to be added.
- **variables.json**: Holds the MQTT broker connection details.

#### Example: devices\_list.json

```
{
  "devices": [
    {
      "name": "a1e7926c-6d35-4b0c-9e8d-023e94c7bc8f",
      "profileName": "Temperature_Humidity_DHT11",
      "serviceName": "device-mqtt",
      "description": "Temperature and humidity sensor",
      "espName": "ESP1",
      "gpios": [ 27 ]
    },
  ]
}
```

- **devices[]**: List of devices to be added
- **name**: name of the device, in this example a uuid
- **profileName**: Device profile associated with the device
- **serviceName**: Device service associated with the device. "device-mqtt" will be used for devices that publish to an mqtt broker.
- **description**: (Optional) short description of the device to be added.
- **espName**: Name of the board on which the device is connected. Useful for systems with multiple ESPs.
- **gpio**: Data pin of the iot-device

#### Example: variables.json

```
{
  "mqttBroker": "mqtt.example.com",
  "mqttPort": 1883,
  "mqttTopicNewDevice": "Devices/ESP1/Config/NewDevice"
}
```

- **mqttBroker**: MQTT broker host
- **mqttPort**: MQTT broker port
- **mqttTopicNewDevice**: Topic used to add new devices, subscribed to by the iot-config-server.

### 1.2.4. Running the Script

To execute the script, use the following command:

```
$ python3 add_device_from_json.py --json path/to/your/devices_list.json
```

- Replace path/to/your/devices\_list.json with the actual path to your JSON file if it is not in the same directory as the script.

Or navigate to the correct directory where your file is stored, and use the following command:

```
$ python3 add_device_from_json.py
```

### 1.2.5. Script Execution

Upon running the script, it connects to the MQTT broker and publishes each device's metadata to the specified topic. The console will display a message for each device added:

```
Added Device: [Device Name]
```

### 1.2.6. Troubleshooting

If you encounter issues, consider the following common problems and solutions:

- **MQTT Connection Failure:** Ensure the MQTT broker address and port are correct in variables.json. Check network connectivity.
- **JSON Format Error:** Verify that the devices\_list.json and variables.json files are properly formatted. Use a JSON validator if necessary.
- **Script Fails to Run:** Make sure all required libraries are installed. Check for typos or errors in the command line used to run the script.

## 1.3. Adding ONVIF Devices Script

This script automates the process of adding ONVIF device profiles to the Universal Gateway via MQTT by publishing the necessary metadata.

### 1.3.1. Script Overview

The `add_device_onvif.py` script is designed to facilitate the addition of ONVIF devices to the Universal Gateway. It reads ONVIF device details from a JSON file and publishes this information to an MQTT broker.

### 1.3.2. File Preparation

You will need two JSON files:

- **device\_onvif.json**: Contains the metadata for the ONVIF devices to be added.
- **variables.json**: Holds the MQTT broker connection details.

Example: `device_onvif.json`

```
{
  "devices": [
    {
      "name": "Cam1",
      "description": "Camera object detection inference",
      "username": "cameraUsername",
      "password": "cameraPassword",
      "credentialsName": "Administrator",
      "authMode": "usernameToken",
      "ipAddress": "192.168.10.110",
      "port": 2020,
      "macAddress": "CE:57:B3:4B:8A:F0"
    }
  ]
}
```

Fields Explained:

- **name**: Unique identifier for the device. This should be a clear and descriptive name.
- **description**: A brief description of the device. This should provide a quick overview of what the device is and its primary functions.
- **username** and **password**: Credentials required for accessing the device.
- **credentialsName**: Label for the set of credentials.
- **authMode**: Authentication mode used by the device.
- **ipAddress**: IP address of the device.
- **port**: Network port on which the device operates.
- **macAddress**: MAC address of the device.

### 1.3.3. Running the Script

To execute the script, use the following command:

```
$ python add_device_onvif.py
```

Ensure that the `device_onvif.json` file is in the same directory as the `add_device_onvif.py` script.

### 1.3.4. Script Execution

Upon running the script, it connects to the MQTT broker and publishes each ONVIF device's metadata to the specified topic. The console will display a message for each device added:

```
Added Device: [Device Name]
```

## 1.4. Adding Stream Script

### 1.4.1. Script Overview

The **`add_stream.py`** script is designed to facilitate the addition of data streams to the Universal Gateway by publishing SQL queries to an MQTT broker. The script uses MQTT to send a message containing an SQL query that defines a new stream configuration. This stream can be used for processing or analyzing data collected by the Gateway.

### 1.4.2. File Preparation

You will need the `variables.json` file configured with the necessary MQTT broker details and the SQL query for stream creation.

#### Example: `variables.json` (Relevant Section)

```
...  
"addStream": {  
  "sql": "create stream demo() WITH (FORMAT=\"JSON\", TYPE=\"edgex\")"  
}  
...
```

#### Fields Explained:

- **sql**: SQL command used to create a new stream. This typically includes the stream's configuration and specifications, such as format and type.

### 1.4.3 Running the Script

To execute the script, use the following command in your command line:

```
python add_stream.py
```

Ensure that the `variables.json` file is in the same directory as the `add_stream.py` script or provide the correct path to it.

### 1.4.4. Script Execution

Upon running the script, it connects to the MQTT broker and publishes the SQL query to the specified topic. This action triggers the creation or modification of a data stream within the Universal Gateway. The success of this operation depends on the proper configuration of both the MQTT broker and the SQL query.

## 1.5. Adding Rule Script

### 1.5.1. Script Overview

The `add_rule.py` script is designed to automate the process of adding rules to the Universal Gateway. It sends a message to an MQTT broker with details of a rule including SQL queries and actions to be performed when the rule conditions are met. This rule can trigger specific actions based on device data, making it an essential part of automation and response within the gateway ecosystem.

### 1.5.2. File Preparation

You need to update the `variables.json` file with the necessary details for MQTT communication and rule definition.

**Example: `variables.json` (Relevant Section)**

```
...  
"addRule": {  
  "ruleName": "doorclosed",  
  "sqlQuery": "SELECT touched FROM demo WHERE touched = true",  
  "deviceName": "b278142d-4f8e-45ea-9e07-dc3561d2810c",  
  "resourceName": "Working",  
  "resourceValue": "false"  
}
```

...

- **ruleName**: The unique identifier for the rule.
- **sqlQuery**: The SQL query that defines the condition under which the rule is triggered.
- **deviceName**: The unique identifier of the device affected by the rule.
- **resourceName**: The specific resource on the device that the rule will interact with.
- **resourceValue**: The value that will be applied to the resource when the rule conditions are met.

### 1.5.3. Running the Script

To execute the script, run the following command:

```
$ python add_rule.py
```

Make sure that the `variables.json` file is in the same directory as the `add_rule.py` script.

### 1.5.4. Script Execution

Upon running the script, it connects to the MQTT broker and publishes the rule configuration to the specified MQTT topic. This process triggers the Universal Gateway to add the rule into its system, enabling it to execute predefined actions based on the specified conditions.

## 1.6. Command Device Script

### 1.6.1. Script Overview

The `command_device.py` script is designed to command a specific device by sending a command message via MQTT. This message adjusts the state of a designated resource on the device according to the provided value, facilitating direct device control through the Universal Gateway.

### 1.6.2. File Preparation

You will need to configure the `variables.json` file with the necessary MQTT and device command details.

### Example: `variables.json` (Relevant Section)

```
...  
"Command": {  
    "deviceName": "ea5a1f69-42c1-4de5-b2a6-8c9b7a8db135",  
    "deviceResource": "Working",  
    "deviceResourceValue": "true"  
}  
...
```

- **deviceName**: The unique identifier of the device to be commanded.
- **deviceResource**: The specific resource on the device that the command will target.
- **deviceResourceValue**: The value to be set for the specified resource.

### 1.6.3. Running the Script

To execute the script, use the following command:

```
$ python command_device.py
```

Ensure that the **`variables.json`** file is in the same directory as the **`command_device.py`** script.

### 1.6.4. Script Execution

Upon running the script, it connects to the MQTT broker and sends a command message to the specified topic. This message contains the command to set the value of a device resource, encoded in Base64 to ensure the data integrity during transmission. The Universal Gateway processes this message to perform the command on the specified device.

## 1.7. Delete Device Script

### 1.7.1. Introduction

The `delete_device.py` script allows you to remove a specific device from the Universal Gateway. This operation is executed by sending a deletion command via MQTT.

### 1.7.2. How to Use

1. Update the variables.json with the device name:

```
"deleteDevice": {  
  "deviceName": "Cam1"  
}
```

2. Run the script using the command:

```
$ python delete_device.py
```

3. The script connects to the MQTT broker and sends a deletion request for the specified device.

## 1.8. Delete All Devices Script

### 1.8.1. Introduction

The `delete_all_devices.py` script enables clearing all devices registered with the Universal Gateway. This can be useful for system resets or extensive configuration changes.

### 1.8.2. How to Use

1. Ensure there are no specific configurations needed in variables.json for deleting all devices.
2. Execute the script using:

```
$ python delete_all_devices.py
```

3. The script publishes a message to the MQTT broker to remove all devices.

## 1.9. Delete Profile Script

### 1.9.1. Introduction

The delete\_profile.py script facilitates the removal of a specific device profile from the Universal Gateway. This helps manage the lifecycle of device profiles effectively.

### 1.9.2 How to Use

1. Confirm that the deviceProfileName to be deleted is correctly specified in variables.json

```
"deleteProfile": {  
  "deviceProfileName": "Camera-Inference"  
}
```

2. Run the script using:

```
$ python delete_profile.py
```

3. It will send a deletion command to the MQTT broker for the specified profile.

### Deleting from the Edge

1. Delete Device.
2. Delete Profile

à The order of these steps is crucial

### ADDING TO the Edge

1. Add Profile.
2. Add Device

à The order of these steps is crucial

## 1.10. Delete Devices by Profile Script

### 1.10.1. Introduction

The `delete_devices_by_profile.py` script removes all devices associated with a particular device profile from the Universal Gateway. Ideal for cleaning up after testing or removing deprecated devices.

### 1.10.2. How to Use

1. Specify the `deviceProfileName` in `variables.json`:

```
"deleteDevicesByProfile": {  
  "deviceProfileName": "onvif-camera"  
}
```

2. Execute the script using:

```
$ python delete_devices_by_profile.py
```

3. The script sends a request to the MQTT broker to delete all devices linked to the specified profile.

## 1.11. Delete Rule Script

### 1.11.1. Introduction

The `delete_rule.py` script allows for the deletion of specific rules within the Universal Gateway, helping maintain an optimized and relevant rule set.

### 1.11.2. How to Use

1. Update the `ruleName` in `variables.json`:

```
"deleteRule": {  
  "ruleName": "doorclosed"  
}
```

2. Run the script using:

```
$ python delete_rule.py
```

3. It publishes a command to the MQTT broker to delete the specified rule.

## 1.12. Delete Stream Script

### 1.12.1. Introduction

The delete\_stream.py script enables the removal of specific data streams from the Universal Gateway, managing data flow efficiently.

### 1.12.2. How to Use

1. Update the streamName that needs to be deleted in variables.json:

```
"deleteStream" : {  
    "streamName" : "demo"  
}
```

2. Execute the script using:

```
$ python delete_stream.py
```

3. The script sends a deletion command for the specified stream to the MQTT broker.

## 2. Compile code in the ESP32 :

### 2.1 Device Factory:

- Verify that the sensors and actuators you are using are declared in devicefactory (source code file).
- Modify Device Factory Code if not declared to add the new device .

### 2.2. Build and upload the project:

Open the PlatformIO Home by clicking on the PlatformIO icon in the Activity Bar.

- In the PlatformIO Home, go to **Project Tasks** under your environment.
- Click on **Build** to compile your code. This will ensure all your code compiles correctly.
- After a successful build, click on **Upload** to upload the code to your ESP32 board.

## 2.3 Monitor the serial output:

- To monitor the serial output from your ESP32, go to **Project Tasks** under your environment.
- Click on **Monitor** to start the serial monitor at the speed specified in platformio.ini

## 2. Tutorial

### 2.1. Adding new Sensor

The Universal Gateway solution is built to manage and control a plethora of devices.

Some steps need to be followed to add a new device for the first time.

In this tutorial, we will be adding an Ultrasonic Sensor HC-SR04 to the Gateway.

#### 2.1.1. Adding Sensor code

As an example, we will need a working code for the device. By searching on google “hr-sr04 esp32 code”, and after choosing the first link in the search result ([ESP32 with HC-SR04 Ultrasonic Sensor with Arduino IDE | Random Nerd Tutorials](#)), we can better understand how the Sensor works and get a working code for the ESP32.

```
const int trigPin = 5;
const int echoPin = 18;

//define sound speed in cm/uS
#define SOUND_SPEED 0.034
#define CM_TO_INCH 0.393701

long duration;
float distanceCm;
float distanceInch;

void setup() {
  Serial.begin(115200); // Starts the serial communication
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
}

void loop() {
  // Clears the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Sets the trigPin on HIGH state for 10 micro seconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Reads the echoPin, returns the sound wave travel time in microseconds
```

```

duration = pulseIn(echoPin, HIGH);

// Calculate the distance
distanceCm = duration * SOUND_SPEED/2;

// Convert to inches
distanceInch = distanceCm * CM_TO_INCH;

// Prints the distance in the Serial Monitor
Serial.print("Distance (cm): ");
Serial.println(distanceCm);
Serial.print("Distance (inch): ");
Serial.println(distanceInch);

delay(1000);
}

```

In the code above we see that we need 2 GPIO pins for the sensor to work. The `setup()` and `loop()` methods will be important for us to create a new class for the Sensor in our ESP32 code.

First, we create the “`ultraSonicSensor.h`” header file:

```

#ifndef ULTRASONICSENSOR_H
#define ULTRASONICSENSOR_H

#include "sensor.h"

class UltraSonicSensor : public Sensor {
public:
    UltraSonicSensor(const std::array<int, 4>& gpios, const String& device_name);

    void setup() override;
    void loop() override;
    void publishToMQTT();

private:
    float _distance;
    float _duration;
    int _trig_pin;
    int _echo_pin;
};

#endif

```

- **#ifndef ULTRASONICSENSOR\_H:** Check if the variable named after the header was defined
- **#define ULTRASONICSENSOR\_H:** Define the variable named after the header
- **#include "sensor.h":** Includes the parent-class header file to allow inheritance.
- **class UltraSonicSensor : public Sensor:** Class definition inherits from Sensor.
- **public:** In this section, we define public methods and variables of the class.

- **UltrasonicSensor(const std::array<int, 4>& gpios, const String& device\_name):** The constructor of the Class, always takes as parameters, a vector array of size 4 containing GPIO pins(int) and the device\_name as a string.
- **void setup() override:** We will override the setup() function depending on the code we found on the internet.
- **void loop() override:** Same as setup.
- **void publishToMQTT():** Method to send the sensor reading to the MQTT broker.
- **private:** Section used to define the private methods and variables.
  - **float \_distance:** Variable used to store the measured distance.
  - **float \_duration:** Variable used to store the travel duration taken by the sound pulse.
  - **int \_trig\_pin:** Variable to store the trigPin
  - **int \_echo\_pin:** Variable to store the echoPin

Now that we have a header file for our class we need to create an “ultraSonicSensor.cpp” implementation file.

Now we will extract the informations we need from the code found on the internet before.

```
const int trigPin = 5;
const int echoPin = 18;

//define sound speed in cm/uS
#define SOUND_SPEED 0.034
#define CM_TO_INCH 0.393701

long duration;
float distanceCm;
float distanceInch;
```

we already have a trigPin and echoPin (\_trig\_pin/\_echo\_pin) variables defined so we can skip these variable definitions.

we should define the speed of sound, in this tutorial we will choose to only have the distance in cm so we just need to define the SPEED\_SOUND variable.

Same as for the pins, the distance and duration variables have been defined in the header file.

so we will begin our ultraSonicSensor.cpp file with the code below:

```
#include "ultraSonicSensor.h"
//define speed of sound in cm per micro-seconds
#define SOUND_SPEED 0.034

// get the global external objects and variables
extern const char* mqtt_topic;
extern PubSubClient client;
extern WiFiClient espClient;
```

```

UltraSonicSensor::UltraSonicSensor(const std::array<int, 4>& gpios, const String&
device_name)
: Sensor(gpios, device_name) {
    _distance = 0;
    _duration = 0;
    _trig_pin = _gpios[0];
    _echo_pin = _gpios[1];
    _lastUpdateMillis = millis();
}

```

- **#include "ultraSonicSensor.h"**: include the header file.
- **#define SOUND\_SPEED 0.034**: Define the speed sound variable like the internet example
- **extern const char\* mqtt\_topic**: Gets the global mqtt\_topic define in the main.ino
- **extern PubSubClient client**: get the global MQTT client created in the main.ino
- **extern WiFiClient espClient**: get the global Wifi client created in the main.ino
- **UltraSonicSensor::UltraSonicSensor(const std::array<int, 4>& gpios, const String& device\_name)**: Implementation of the constructor.
  - **Sensor(gpios, device\_name)**: call the parent class constructor to instantiate the variables \_gpios (Array of pins) and device\_name.
  - **\_distance = 0**: Initialize the variable.
  - **\_duration = 0**: Initialize the variable.
  - **\_trig\_pin = \_gpios[0]**: assign the first value of the array \_gpios to the trig pin variable. This an important choice, as when we want to add a device of the type Ultrasonic Sensor to the universal gateway the order of pins in the gpios array matters.
  - **\_echo\_pin = \_gpios[1]**: Assign the second value of the array \_gpios to the echo pin variable.
  - **\_lastUpdateMillis = millis()**: instantiate the \_lastUpdateMillis to the current time in milliseconds.

Now for the setup() loop below:

```

void setup() {
    Serial.begin(115200); // Starts the serial communication
    pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
    pinMode(echoPin, INPUT); // Sets the echoPin as an Input
}

```

The Serial communication doesn't need to be started here because the main.ino handles it. We will set the pin Modes by calling the variables we instantiated in the constructor.

```
void UltraSonicSensor::setup() {
    pinMode(_trig_pin, OUTPUT); // Set the first gpio in the array as an OUTPUT
    (trigPin)
    pinMode(_echo_pin, INPUT); // Set the second gpio in the array as an INPUT
    (echoPin)
}
```

As the class is defined in the header file, every function definition in the implementation file should follow the pattern: "type className::methodName(args) {"

now we move on to the loop():

```
void loop() {
    // Clears the trigPin
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // Sets the trigPin on HIGH state for 10 micro seconds
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Reads the echoPin, returns the sound wave travel time in microseconds
    duration = pulseIn(echoPin, HIGH);

    // Calculate the distance
    distanceCm = duration * SOUND_SPEED/2;

    // Convert to inches
    distanceInch = distanceCm * CM_TO_INCH;

    // Prints the distance in the Serial Monitor
    Serial.print("Distance (cm): ");
    Serial.println(distanceCm);
    Serial.print("Distance (inch): ");
    Serial.println(distanceInch);

    delay(1000);
}
```

we will keep the digitalWrite and pulseIn invocations as is and only change the variables called. Same for distanceCm to \_distance, and remove the inches conversion.

We will not print the result but just call the function that will publish the distance to the MQTT broker.

Below is the resulting loop() method of this class

```
void UltraSonicSensor::loop() {
    unsigned long currentMillis = millis();
    if (currentMillis - _lastUpdateMillis < 2000) {
        return;
    } // non-blocking delay
```

```

        _lastUpdateMillis = millis();

        // Clears the trigPin
        digitalWrite(_trig_pin, LOW);
        delayMicroseconds(2);
        // Sets the trigPin on HIGH state for 10 microseconds
        digitalWrite(_trig_pin, HIGH);
        delayMicroseconds(10);
        digitalWrite(_trig_pin, LOW);

        // Reads the echoPin, returns the sound wave travel time in microseconds
        _duration = pulseIn(_echo_pin, HIGH);

        _distance = _duration * SOUND_SPEED/2;

        publishToMQTT();
    }

```

We first implement a non-blocking delay as the ESP32 will be managing multiple devices, here we chose a 2000ms delay. Then we modified the variables used.

Finally the publishToMQTT() method:

```

void UltraSonicSensor::publishToMQTT() {
    publishSingleValue(_distance, "distance");
}

```

The second argument, "distance", is crucial. When creating a device profile for the sensor, the deviceResource name must match this exact value. The Universal Gateway relies on this deviceResource name to correctly read the associated value.

The final result for "ultraSonicSensor.cpp":

```

#include "ultraSonicSensor.h"
//define speed of sound in cm per micro-seconds
#define SOUND_SPEED 0.034

extern const char* mqtt_topic;
extern PubSubClient client;
extern WiFiClient espClient;

UltraSonicSensor::UltraSonicSensor(const std::array<int, 4>& gpios, const String& device_name)
    : Sensor(gpios, device_name) {
    _distance = 0;
    _duration = 0;
    _trig_pin = _gpios[0];
    _echo_pin = _gpios[1];
    _lastUpdateMillis = millis();
}

```

```

    }

void UltraSonicSensor::setup() {
    pinMode(_trig_pin, OUTPUT); // Set the first gpio in the array as an OUTPUT
    (trigPin)
    pinMode(_echo_pin, INPUT); // Set the second gpio in the array as an INPUT
    (echoPin)
}

void UltraSonicSensor::loop() {
    unsigned long currentMillis = millis();
    if (currentMillis - _lastUpdateMillis < 2000) {
        return;
    } // non-blocking delay

    _lastUpdateMillis = millis();
    // Clears the trigPin
    digitalWrite(_trig_pin, LOW);
    delayMicroseconds(2);
    // Sets the trigPin on HIGH state for 10 microseconds
    digitalWrite(_trig_pin, HIGH);
    delayMicroseconds(10);
    digitalWrite(_trig_pin, LOW);

    // Reads the echoPin, returns the sound wave travel time in microseconds
    _duration = pulseIn(_echo_pin, HIGH);

    _distance = _duration * SOUND_SPEED/2;

    publishToMQTT();
}

void UltraSonicSensor::publishToMQTT() {
    publishSingleValue(_distance, "distance");
}

```

At this point the class is ready we just need to add it to the deviceFactory.

In “deviceFactory.h” we need to include the “ultraSonicSensor.h”

```

#ifndef DEVICEFACTORY_H
#define DEVICEFACTORY_H

#include "device.h"
#include "sensor.h"
#include "moistureSensor.h"
#include "temperatureHumiditySensor.h"
#include "relayActuator.h"
#include "limitSwitch.h"
#include "temperatureSensor.h"
#include "ultraSonicSensor.h" // <-----

```

```

class DeviceFactory {
public:
    static Device* createDevice(const String& profileName, const std::array<int, 4>&
gpios, const String& device_name);
};

#endif

```

Then in the deviceFactory.cpp add an else if to call the new sensor's constructors:

```

} else if (profileName == "Ultra_Sonic_Sensor") {
    Serial.println("Created an Ultrasonic Sensor name:" + device_name);
    return new UltraSonicSensor(gpios, device_name);
}

```

The profileName needs to be the same as the one created in the next part.

## 2.1.2. Adding Device Profile

Now we need to create a device profile so that the UniversalGateway understand how to interact with the device. In this example the Universal Gateway needs to read the distance measured. Below is the device\_profile.json [seen before](#).

```

{
  "deviceProfiles": [
    {
      "name": "Ultra_Sonic_Sensor",
      "manufacturer": "XXX",
      "model": "HC-SR04",
      "labels": [
        "esp32",
        "sensor",
        "mqtt",
        "ultrasonic",
        "distance"
      ],
      "description": "Ultrasonic distance Sensor",
      "deviceResources": [
        {
          "name": "distance",
          "isHidden": false,
          "description": "Distance in cm",
          "properties": {
            "valueType": "Float32",
            "readWrite": "R",
            "units": "cm"
          }
        }
      ]
    }
  ]
}

```

```
} ]
```

### Important Fields:

- **"name": "Ultra\_Sonic\_Sensor"**: The name of the deviceProfile.
- **"deviceResources": [** : Array of deviceResources
  - **"name": "distance"**: Name of the deviceResource to be created (same as the one mentioned in the ESP32 before.
  - **"isHidden": false**: keep it false as long as you need to see the measurement associated with the deviceResource
  - **"properties": {**: Properties of the device resource
    - **"valueType": "Float32"**: Type of data being sent to the Universal Gateway
    - **"readWrite": "R"**: The value of the device is readable, can also be writable or both (R, W, or RW)
    - **"units": "cm"**: units of the value.

### 2.1.3. Adding Device

After Adding the Device Profile, Now we can add one or more Ultrasonic Sensor. Below is an example of a json object that can be added to the [devices\\_list.json](#).

```
{
  "name": "ea5a1f69-42c1-4de5-b2a6-8c9b7a8db126",
  "profileName": "Ultra_Sonic_Sensor",
  "serviceName": "device-mqtt",
  "description": "hcsr04 sensor",
  "espName": "ESP1",
  "gpios": [25,34]
}
```

The "profileName" is the one we just created in before. serviceName should always be "device-mqtt" as the esp32 published data to the mqtt broker. The order of gpios in the array is important for this type of device that needs more than 1 pin (not counting Vcc and GND).